Universidad de San Carlos de Guatemala

Facultad de Ingeniería
Curso
Sección:
Ing.

Tutor Académico:

Proyecto 2

Nombre: Diego Alberto Maldonado Gálvez

Carné: 202200092

Fecha: 20/04/2026

Desarrollo de

Introducción:

Este documento detalla la estructura interna del intérprete para el lenguaje GoScript. A diferencia de un compilador tradicional que traduce a código máquina, este proyecto es un Intérprete, lo que significa que lee, analiza y ejecuta las instrucciones en tiempo real.

El proyecto se construyó bajo una arquitectura de Cliente-Servidor. El cliente es una interfaz web pura y el servidor es una API REST construida en TypeScript con Node.js. Para lograr que la computadora entienda el texto del usuario, se utilizó la herramienta Jison, la cual ayuda a generar las reglas gramaticales para desarmar el código.

Herramientas Utilizadas:

TypeScript y Node.js: Lenguaje y entorno base para toda la lógica de programación del servidor.

Jison: Herramienta utilizada para generar el Analizador Léxico (que reconoce las palabras válidas) y el Analizador Sintáctico (que valida el orden correcto de las palabras).

Express: Librería utilizada para levantar el servidor y comunicar la interfaz web con la lógica del intérprete de forma local.

HTML/JS/CSS: Utilizados para construir la interfaz visual del usuario.

D3-Graphviz: Librería utilizada para dibujar el Árbol de Sintaxis Abstracta (AST) de forma gráfica.

Diccionario de Métodos y Clases:

Clase Entorno: Es la memoria del programa. Guarda una tabla hash (diccionario) con los nombres de las variables y sus valores actuales. Sabe diferenciar entre variables globales y variables que solo existen dentro de un bloque específico (Shadowing).

```
src > entorno > Entorno.ts > Entorno > actualizar
import { Simbolo } from './Simbolo';

export class Entorno {
    public anterior: Entorno | null;
    public tabla: Map<string, Simbolo>;

    constructor(anterior: Entorno | null) {
        this.anterior = anterior;
        this.tabla = new Map<string, Simbolo>();
    }

    public guardar(id: string, simbolo: Simbolo): boolean {
        if (this.tabla.has(id)) {
            return false;
        }
        this.tabla.set(id, simbolo);
        return true;
    }

    public obtener(id: string): Simbolo | null {
        let entornoActual: Entorno | null = this;

        while (entornoActual != null) {
            if (entornoActual.tabla.has(id)) {
                return entornoActual.tabla.get(id) || null;
            }
            // Si no esta aca, busco en el entorno superior
            entornoActual = entornoActual.anterior;
        }
        return null;
    }

    public actualizar(id: string, simbolo: Simbolo): boolean {
        let entornoActual: Entorno | null = this;

        while (entornoActual != null) {
            if (entornoActual.tabla.has(id)) {
                entornoActual.tabla.set(id, simbolo);
                return true;
            }
            entornoActual = entornoActual.anterior;
        }

        return false;
    }


}
```

Interfaz Instrucción: Es el molde principal. Obliga a que cualquier cosa que el usuario escriba (un if, un print, una suma) tenga que saber cómo "interpretarse" a sí misma.

```
export interface Instruccion {

    // El numero de linea donde se encuentra la instruccion util para reportar errores
    linea: number;

    //La columna donde se encuentra la instruccion
    columna: number;

    /**
     * Metodo principal que ejecutara la logica de este nodo
     * @param entorno La tabla de simbolos actual, variables, funciones, etc
     * @param arbol El objeto global que guarda la consola, errores, etc
     */
    interpretar(entorno: any, arbol: any): any;
}
```

Nodos de Operación (Aritmética, Lógica, Relacional): Clases que toman dos valores, revisan que los tipos de datos sean compatibles y devuelven un resultado matemático o un booleano.

Aritmetica.ts

src > ast > Aritmetica.ts > Aritmetica > interpretar

```
import { Instruccion } from './Instruccion';
import { Entorno } from '../entorno/Entorno';
import { TipoDato } from '../entorno/Tipo';

export enum OperadorAritmetico {
    SUMA,
    RESTA,
    MULTIPLICACION,
    DIVISION,
    MODULO,
    NEGACION_UNARIA
}

export class Aritmetica implements Instruccion {
    public linea: number;
    public columna: number;
    public operador: OperadorAritmetico;
    public expresionIzquierda: Instruccion;
    public expresionDerecha: Instruccion | null;

    constructor(linea: number, columna: number, operador: OperadorAritmetico, expIzq: Instruccion, expDer: Instruccion | null = null) {
        this.linea = linea;
        this.columna = columna;
        this.operador = operador;
        this.expresionIzquierda = expIzq;
        this.expresionDerecha = expDer;
    }

    interpretar(entorno: Entorno, arbol: any): any {
        const valorIzq = this.expresionIzquierda.interpretar(entorno, arbol);
        let valorDer = null;

        if (this.expresionDerecha) {
            valorDer = this.expresionDerecha.interpretar(entorno, arbol);
        }

        let operarComoBool = false;
        if (valorIzq?.tipo === TipoDato.BOOL && valorDer?.tipo === TipoDato.BOOL) {
            operarComoBool = true;
        }

        if (valorIzq && valorIzq.tipo === TipoDato.RUNE) valorIzq.tipo = TipoDato.INT;
        if (valorDer && valorDer.tipo === TipoDato.RUNE) valorDer.tipo = TipoDato.INT;

        if (valorIzq && valorIzq.tipo === TipoDato.BOOL) {
            valorIzq.valor = valorIzq.valor ? 1 : 0;
            valorIzq.tipo = TipoDato.INT;
        }
        if (valorDer && valorDer.tipo === TipoDato.BOOL) {
            valorDer.valor = valorDer.valor ? 1 : 0;
            valorDer.tipo = TipoDato.INT;
        }

        const tipoIzq = valorIzq?.tipo;
        const tipoDer = valorDer?.tipo;
        const valIzq = valorIzq?.valor;
        const valDer = valorDer?.valor;

        let resultado = null;

        switch (this.operador) {
            case OperadorAritmetico.SUMA:
                resultado = this.operarSuma(valIzq, valDer, tipoIzq, tipoDer, arbol); break;
            case OperadorAritmetico.RESTA:
                resultado = this.operarResta(valIzq, valDer, tipoIzq, tipoDer, arbol); break;
            case OperadorAritmetico.MULTIPLICACION:
                resultado = this.operarMultiplicacion(valIzq, valDer, tipoIzq, tipoDer, arbol); break;
            case OperadorAritmetico.DIVISION:
                resultado = this.operarDivision(valIzq, valDer, tipoIzq, tipoDer, arbol); break;
            case OperadorAritmetico.MODULO:
                resultado = this.operarModulo(valIzq, valDer, tipoIzq, tipoDer, arbol); break;
            case OperadorAritmetico.NEGACION_UNARIA:
                resultado = this.operarNegacion(valIzq, tipoIzq, arbol); break;
            default:
                arbol.agregarError("Semantico", "Operador aritmetico desconocido", this.linea, this.columna);
```

Logica.ts

src > ast > Logica.ts > Logica > expresionIzquierda

```
import { Instruccion } from './Instruccion';
import { Entorno } from '../entorno/Entorno';
import { TipoDato } from '../entorno/Tipo';

export enum OperadorLogico {
    AND, OR, NOT
}

export class Logica implements Instruccion {
    public linea: number;
    public columna: number;
    public operador: OperadorLogico;
    public expresionIzquierda: Instruccion;
    public expresionDerecha: Instruccion | null;

    constructor(linea: number, columna: number, operador: OperadorLogico, expIzq: Instruccion, expDer: Instruccion | null = null) {
        this.linea = linea;
        this.columna = columna;
        this.operador = operador;
        this.expresionIzquierda = expIzq;
        this.expresionDerecha = expDer;
    }

    interpretar(entorno: Entorno, arbol: any): any {
        const izq = this.expresionIzquierda.interpretar(entorno, arbol);

        if (this.operador === OperadorLogico.NOT) {
            return { valor: !izq.valor, tipo: TipoDato.BOOL };
        }

        const der = this.expresionDerecha?.interpretar(entorno, arbol);

        let resultado = false;
        if (this.operador === OperadorLogico.AND) {
            resultado = (izq.valor && der.valor);
        } else if (this.operador === OperadorLogico.OR) {
            resultado = (izq.valor || der.valor);
        }

        return { valor: resultado, tipo: TipoDato.BOOL };
    }
}
```

Relacional.ts

src > ast > Relacional.ts > Relacional > constructor

```
import { Instruccion } from './Instruccion';
import { Entorno } from '../entorno/Entorno';
import { TipoDato } from '../entorno/Tipo';

export enum OperadorRelacional {
    IGUAL, DISTINTO, MAYOR, MENOR, MAYOR_IGUAL, MENOR_IGUAL
}

export class Relacional implements Instruccion {
    public linea: number;
    public columna: number;
    public operador: OperadorRelacional;
    public expresionIzquierda: Instruccion;
    public expresionDerecha: Instruccion;

    constructor(linea: number, columna: number, operador: OperadorRelacional, expIzq: Instruccion, expDer: Instruccion) {
        this.linea = linea;
        this.columna = columna;
        this.operador = operador;
        this.expresionIzquierda = expIzq;
        this.expresionDerecha = expDer;
    }

    interpretar(entorno: Entorno, arbol: any): any {
        const izq = this.expresionIzquierda.interpretar(entorno, arbol);
        const der = this.expresionDerecha.interpretar(entorno, arbol);

        const valIzq = izq.valor;
        const valDer = der.valor;

        let resultado = false;

        switch (this.operador) {
            case OperadorRelacional.IGUAL: resultado = (valIzq === valDer); break;
            case OperadorRelacional.DISTINTO: resultado = (valIzq !== valDer); break;
            case OperadorRelacional.MAYOR: resultado = (valIzq > valDer); break;
            case OperadorRelacional.MENOR: resultado = (valIzq < valDer); break;
            case OperadorRelacional.MAYOR_IGUAL: resultado = (valIzq >= valDer); break;
            case OperadorRelacional.MENOR_IGUAL: resultado = (valIzq <= valDer); break;
        }

        return { valor: resultado, tipo: TipoDato.BOOL };
    }
}
```

Desarrollo de

Clases de Control (If, For, Switch): Controlan el flujo. Tienen la inteligencia lógica para decidir si ejecutan un bloque de código o se lo saltan dependiendo de una condición. También manejan las detenciones de ciclos o funciones como break, continue o return.

If.ts

src > ast > If.ts > ...

```
import { Instruccion } from './Instruccion';
import { Entorno } from '../entorno/Entorno';
import { TipoDato } from '../entorno/Tipo';

export class If implements Instruccion {
    public linea: number;
    public columna: number;
    public condicion: Instruccion;
    public instruccionesIf: Instruccion[];
    public instruccionesElse: Instruccion[] | Instruccion | null;

    constructor(linea: number, columna: number, condicion: Instruccion, instruccionesIf: Instruccion[], instruccionesElse: Instruccion[] | Instruccion | null = null) {
        this.linea = linea;
        this.columna = columna;
        this.condicion = condicion;
        this.instruccionesIf = instruccionesIf;
        this.instruccionesElse = instruccionesElse;
    }

    interpretar(entorno: Entorno, arbol: any): any {
        const cond = this.condicion.interpretar(entorno, arbol);

        if (cond.tipo !== TipoDato.BOOL) {
            arbol.agregarError("Semantico", "La condicion del IF debe ser de tipo BOOL", this.linea, this.columna);
            return;
        }

        if (cond.valor === true) {
            const entornoLocal = new Entorno(entorno);
            for (const instruccion of this.instruccionesIf) {
                const res = instruccion.interpretar(entornoLocal, arbol);
                if (res !== null && res !== undefined) return res;
            }
        } else if (this.instruccionesElse !== null) {
            if (this.instruccionesElse instanceof If) {
                return this.instruccionesElse.interpretar(entorno, arbol);
            } else {
                const entornoLocal = new Entorno(entorno);
                for (const instruccion of this.instruccionesElse as Instruccion[]) {
                    const res = instruccion.interpretar(entornoLocal, arbol);
                    if (res !== null && res !== undefined) return res;
                }
            }
        }
    }
}
```

For.ts

src > ast > For.ts > For > interpretar

```
import { Instruccion } from './Instruccion';
import { Entorno } from '../entorno/Entorno';
import { TipoDato } from '../entorno/Tipo';

export class For implements Instruccion {
    public linea: number;
    public columna: number;
    public condicion: Instruccion;
    public instrucciones: Instruccion[];
    public inicializacion: Instruccion | null;
    public incremento: Instruccion | null;

    constructor(linea: number, columna: number, condicion: Instruccion, instrucciones: Instruccion[], inicializacion: Instruccion | null = null, incremento: Instruccion | null = null) {
        this.linea = linea;
        this.columna = columna;
        this.condicion = condicion;
        this.instrucciones = instrucciones;
        this.inicializacion = inicializacion;
        this.incremento = incremento;
    }

    interpretar(entorno: Entorno, arbol: any): any {
        const entornoFor = new Entorno(entorno);

        if (this.inicializacion) {
            this.inicializacion.interpretar(entornoFor, arbol);
        }

        let cond = this.condicion.interpretar(entornoFor, arbol);
        if (cond?.tipo !== TipoDato.BOOL) {
            arbol.agregarError("Semantico", "La condicion del for debe ser BOOL", this.linea, this.columna);
            return;
        }

        while (cond.valor === true) {
            const entornoIteracion = new Entorno(entornoFor);
            let salir = false;

            for (const instruccion of this.instrucciones) {
                const resultado = instruccion.interpretar(entornoIteracion, arbol);

                if (resultado?.tipo === 'BREAK') {
                    salir = true;
                    break;
                }
                if (resultado?.tipo === 'CONTINUE') {
                    break;
                }
            }

            if (salir) break;

            if (this.incremento) {
                this.incremento.interpretar(entornoFor, arbol);
            }

            cond = this.condicion.interpretar(entornoFor, arbol);
        }
    }
}
```

Desarrollo de

```
Switch.ts
src > ast > Switch.ts > Switch > interpretar
import { Instruccion } from './Instruccion';
import { Entorno } from '../entorno/Entorno';
import { Caso } from './Caso';

export class Switch implements Instruccion {
    public linea: number;
    public columna: number;
    public expresion: Instruccion;
    public casos: Caso[];

    constructor(linea: number, columna: number, expresion: Instruccion, casos: Caso[]) {
        this.linea = linea;
        this.columna = columna;
        this.expresion = expresion;
        this.casos = casos;
    }

    interpretar(entorno: Entorno, arbol: any): any {
        const valorSwitch = this.expresion.interpretar(entorno, arbol);
        let casoEjecutado = false;

        //se busca coincidencias en los "case"
        for (const caso of this.casos) {
            if (caso.expresion !== null) {
                const valorCaso = caso.expresion.interpretar(entorno, arbol);

                if (valorSwitch.valor === valorCaso.valor && valorSwitch.tipo === valorCaso.tipo) {
                    const entornoLocal = new Entorno(entorno);
                    for (const instruccion of caso.instrucciones) {
                        const res = instruccion.interpretar(entornoLocal, arbol);
                        if (res?.tipo === 'BREAK' || res?.tipo === 'CONTINUE' || res?.tipo === 'RETURN') return res;
                    }
                    casoEjecutado = true;
                    break;
                }
            }
        }

        //si no hubo coincidencia se busca si existe un "default"
        if (!casoEjecutado) {
            for (const caso of this.casos) {
                if (caso.expresion === null) { // Este es el default
                    const entornoLocal = new Entorno(entorno);
                    for (const instruccion of caso.instrucciones) {
                        const res = instruccion.interpretar(entornoLocal, arbol);
                        if (res?.tipo === 'BREAK' || res?.tipo === 'CONTINUE' || res?.tipo === 'RETURN') return res;
                    }
                    break;
                }
            }
        }
    }
}
```

Estructuras Complejas (Slices y Structs): Clases especializadas que manejan agrupaciones de datos. Estas clases están programadas para pasar sus datos "por referencia", lo que significa que si se modifican dentro de una función, su valor original global también cambia.

```
Slice.ts
src > ast > Slice.ts > Slice > interpretar

export class Slice implements Instruccion {
    public linea: number;
    public columna: number;
    public tipoArreglo: TipoDato;
    public listaValores: Instruccion[];

    constructor(linea: number, columna: number, tipoArreglo: TipoDato, listaValores: Instruccion[]) {
        this.linea = linea;
        this.columna = columna;
        this.tipoArreglo = tipoArreglo;
        this.listaValores = listaValores;
    }

    interpretar(entorno: Entorno, arbol: any): any {
        let arreglo: any[] = [];

        for (let exp of this.listaValores) {
            let resultado = exp.interpretar(entorno, arbol);

            if (resultado.tipo !== this.tipoArreglo) {
                if (this.tipoArreglo === TipoDato.FLOAT && resultado.tipo === TipoDato.INT) {
                    resultado.tipo = TipoDato.FLOAT;
                } else {
                    arbol.agregarError("Semantico", "Dato incorrecto para el Slice", this.linea, this.columna);
                    return { valor: null, tipo: TipoDato.NULO };
                }
            }
            arreglo.push(resultado.valor);
        }

        return { valor: arreglo, tipo: this.tipoArreglo };
    }
}
```

AccesoStruct.ts

src > ast > AccesoStruct.ts > AccesoStruct

```typescript
import { Instruccion } from './Instruccion';
import { Entorno } from '../entorno/Entorno';
import { TipoDato } from '../entorno/Tipo';

export class AccesoStruct implements Instruccion {
    constructor(public linea: number, public columna: number, public id: string, public atributo: string) {}

    interpretar(entorno: Entorno, arbol: any): any {
        const simbolo = entorno.obtener(this.id);

        if (!simbolo || simbolo.tipo !== TipoDato.STRUCT) {
            arbol.agregarError("Semantico", "El Struct '" + this.id + "' no existe o no es un Struct", this.linea, this.columna);
            return { valor: null, tipo: TipoDato.NULO };
        }

        const instancia = simbolo.valor;

        if (!(this.atributo in instancia)) {
            arbol.agregarError("Semantico", "El atributo '" + this.atributo + "' no existe en el struct", this.linea, this.columna);
            return { valor: null, tipo: TipoDato.NULO };
        }

        return { valor: instancia[this.atributo], tipo: TipoDato.ANY };
    }
}
```

Desarrollo de

Manejo y Recuperación de Errores:

El sistema implementa una recuperación silenciosa (Panic Mode Recovery a nivel léxico). Si el usuario se equivoca, el programa atrapa el error, lo guarda y continúa leyendo para encontrar más fallos:

- **Errores Léxicos:** Atrapados por Jison cuando el usuario escribe un símbolo que no existe en el lenguaje (ej. @, $, ~). Se guardan en una variable global.
- **Errores Sintácticos:** Atrapados cuando el usuario escribe algo fuera del orden permitido.
- **Errores Semánticos:** Atrapados por TypeScript en tiempo de ejecución (ej. intentar sumar una palabra con un booleano). Todos estos se empaquetan en formato JSON y se envían a la interfaz web para llenar la Tabla de Errores indicando la línea y columna exacta.

Conclusiones

- Se logró implementar un intérprete robusto que separa la lectura del texto de la lógica de ejecución mediante el uso del patrón de diseño AST.
- Manejar el flujo de las sentencias (como el "burbujeo" de los return dentro de la recursividad) requirió un diseño cuidadoso para evitar desbordamientos de memoria (Maximum call stack size exceeded).
- La integración de un frontend web puro conectado a una API REST en Node.js demuestra que se puede construir un IDE completamente funcional, estético y ligero sin depender de aplicaciones de escritorio pesadas.

-